

Data Flywheel for Autonomous Systems

Design for off-road perception at fleet scale

Nakshatra Vyas — Data Engineer, Autonomous Systems

Zensomy Autonomous Technologies · October 2026

Ingestion · selection · training · release · the three edges that close the loop



Two numbers decide the architecture

178 h

of upload for 8 hours of driving

400 GB per vehicle-day against a 5 Mbps rural uplink. At \$1–10/GB that is \$400–4,000 per vehicle per day.

~1%

of recorded footage can be annotated

10,000 hours recorded against roughly 100 hours of annotation budget. The limit is financial, so more compute does not relieve it.

Streaming everything to the cloud is not a rejected trade-off. It is impossible. And labelling everything that does arrive is impossible too.



A pipeline runs and stops. A flywheel stores momentum

PIPELINE

New data arrives. The same steps run again. Each run costs what the last one cost and buys what the last one bought.

FLYWHEEL

Each turn makes the next turn cheaper and more productive, because information flows backwards as well as forwards.

The feedback edges are the design. Remove them and this is a pipeline that happens to be run repeatedly.

Three edges, named and mechanised: field failures fire retraining · weak evaluation slices steer the next collection · shadow-mode disagreements become annotation candidates.

The whole loop, one picture



↑ Three dotted edges run backwards from release and evaluation into training and selection — **solid lines are data, dotted lines are decisions.**

The vehicle decides what leaves, in four tiers

1 · CELLULAR TELEMETRY

Always on. Kilobytes per second.

2 · CELLULAR CLIPS

Flagged events only.

3 · DEPOT WIFI

Bulk — about **98% of bytes**, at near-zero marginal cost.

4 · PHYSICAL MEDIA

Sites with no usable link.

- **Durable local buffer, store-and-forward** — a cloud or link outage delays ingestion, it does not lose data. The most important reliability property in the system lives on the vehicle, not in the cloud.
- **Priority queue, not FIFO** — when the buffer fills or a link returns briefly, flagged data goes first.
- **Idempotent keys and resumable multipart upload** — over unreliable links, unknown completion state is the normal case.

Cost Tier 3 depends on machines returning to a depot most days. That is the most load-bearing assumption in the design, and tier 4 is its explicit fallback.

Integrity and quality are two different problems

INTEGRITY — HARD FAIL

Did all the bytes arrive intact? Manifest reconciliation, SHA-256 per chunk, chunk count, sequence continuity.

→ **quarantine/**, 90 days for diagnosis

QUALITY — SOFT FLAG

Is the data usable? Structural, temporal, signal-level, semantic.

→ **raw/**, ingested and flagged, visible to selection

A truncated upload must never become canonical data. A dusty lens must never be thrown away — it is exactly the material selection wants.

The silent poisoner

Calibration version is a mandatory field at ingest. A mount that shifts two centimetres without re-calibration invalidates every LiDAR-to-camera projection from that moment on. Nothing fails. No error is raised. Models trained on the mixture degrade and the cause is untraceable unless the version was recorded.

Four signals, because uncertainty cannot see confident error

WRONG · 0.35

Disengagement, emergency brake, near-miss, re-plan storm. **Needs no model.**

UNSURE · 0.30

Low confidence, narrow margin, detection instability.

CONFLICT · 0.20

Camera and LiDAR disagree. Gated on clock sync under 10 ms.

RARE · 0.15

Coverage gap against the scenario taxonomy.

The model that is 97% certain and wrong is the highest-risk failure mode, because the machine proceeds at full speed into it. Only **WRONG** and **CONFLICT** can see it.

- **WRONG also solves the cold start** — a disengagement is a trained operator judging the machine unsafe, and it needs no model at all.
- **CONFLICT is gated** — at 200 ms of skew, correctly functioning sensors appear to disagree on every frame.

Cost Four signals over classic uncertainty sampling: more scoring machinery to build and maintain.

Scoring, diversity, and the budget

$$\text{score} = (0.35 \cdot \text{WRONG} + 0.30 \cdot \text{UNSURE} + 0.20 \cdot \text{CONFLICT} + 0.15 \cdot \text{RARE}) \times \text{quality} \div \text{similarity}$$

CLIP	WRONG	UNSURE	CONFLICT	RARE	QUAL	SIM	SCORE	OUTCOME
veh03 disengagement, dust	1.00	0.71	0.44	0.92	0.95	1.0	0.750	Must-take
veh04 LiDAR sees it, camera does not	0.80	0.12	0.78	0.55	0.93	1.0	0.516	Selected
veh03 same dust, 90 s later	1.00	0.68	0.41	0.92	0.95	2.4	0.310	Deferred
veh01 straight line, clear day	0.00	0.08	0.05	0.05	1.00	1.0	0.042	Random 10%

Budget: 20% must-take · 70% scored and diversity-attenuated · 10% random · 15% per-vehicle cap



Two structural choices in that score

Quality multiplies, it does not subtract

A faulty sensor produces high uncertainty **and** high cross-sensor conflict at once. An additive penalty gets outvoted by four strong signals. A multiplier suppresses the clip regardless of how strongly it scores elsewhere — and raises a maintenance alert instead.

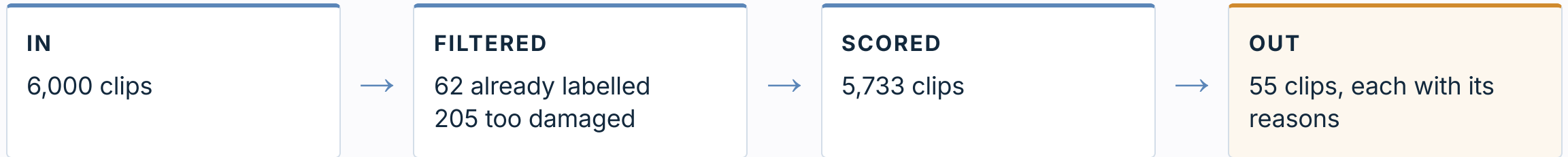
The shortlist is three groups, not a top-N

At shortlist time only WRONG and RARE have been computed. Ranking on those admits almost exclusively event-bearing clips and **systematically excludes what UNSURE and CONFLICT exist to find**. The funnel becomes self-confirming.

10% The random reservation preserves evaluation validity. If every labelled clip was chosen because the model struggled, normal-condition performance becomes unmeasurable — and it is the only mechanism that can surface a failure mode no signal was designed to detect.

Cost A larger shortlist means higher GPU cost at the scoring stage · 10% of the budget is not optimised for information value.

The selection component is implemented



- **73 tests, and each name is a promise the design makes.**
`test_uncertainty_alone_would_invert_the_ranking` proves the four-signal argument by showing that uncertainty alone ranks the dangerous clip **last**.
- **Nothing is dropped silently.** Every skipped clip carries a recorded reason, every limit that bound is named in the report, and a test asserts `input = dropped + scored`.
- **A missing signal is not a zero signal.** With no model yet, UNSURE returns nothing and the remaining signals are reweighted — scoring it zero would punish a clip for which subsystems happened to be reporting.

Two dependencies — numpy and pydantic. `docker compose up --build`, and the run is deterministic: ties break on clip ID, and the manifest hashes the input.

A dataset version is a manifest, not a folder

IMMUTABLE MANIFEST

References plus content hashes, written once. A folder is mutable — someone removes a clip, corrects fifty labels, appends a batch, and the name stays `v4`.

STICKY SESSION SPLITS

Assigned by `hash(session_id)`, never re-drawn. Consecutive frames are near-duplicates, so a random frame split puts a frame in train and its twin in test.

INDEPENDENT LEAKAGE CHECK

Compares **content** — exact hashes plus near-duplicate embeddings — not identifiers. A session re-ingested under a new ID is one drive with two names.

A leaky split does not present as an error. It presents as success: the score comes back high, stable, and unrelated to field performance.

Cost Sticky splits are less data-efficient and cannot be balanced precisely — but re-drawing them invalidates every historical model comparison, and that loss is unrecoverable.

Reproducibility: four pins, and an honest claim

DATASET HASH

GIT SHA

IMAGE DIGEST

not a tag

CONFIG + SEEDS

- **The environment pin is the one usually weakened.** An image tag is a mutable pointer — `:latest` resolves differently over time, and CUDA, driver and library versions shift with no record in the run.
- **Reproducibility is statistical, not bitwise, and we say so.** Bit-identical reproduction costs roughly 20–30% throughput for a property rarely needed.
- **Every run is recorded automatically** — inputs, metrics, artifacts, lineage. A run that is not recorded did not happen.

Claiming stronger reproducibility than has been paid for is worse than claiming none, because someone will rely on it.

Evaluation gates per slice, never on the aggregate

AGGREGATE

+1.2

points overall. Looks like a clear improvement.

DUST SLICE

-5.7

points. For a machine in a dry field, that model is worse.

A scenario that is 3% of the evaluation set contributes 3% of the aggregate, so a severe regression confined to it is invisible — and the rare slices are exactly the costly ones.

- **A frozen evaluation set**, refreshed quarterly and deliberately. A growing exam means successive models sat different tests.
- **A regression suite of real past field incidents, which only grows.** Entries are never removed, because the conditions that produced them have not stopped existing.

Cost More evaluation compute, and occasional noisy failures on small slices — managed with minimum slice sizes and tolerance bands.

The release unit is a bundle, never a model alone

SOFTWARE IMAGE

COMPILED MODELS

CONFIG

CALIBRATION
SCHEMA

HARDWARE TARGET

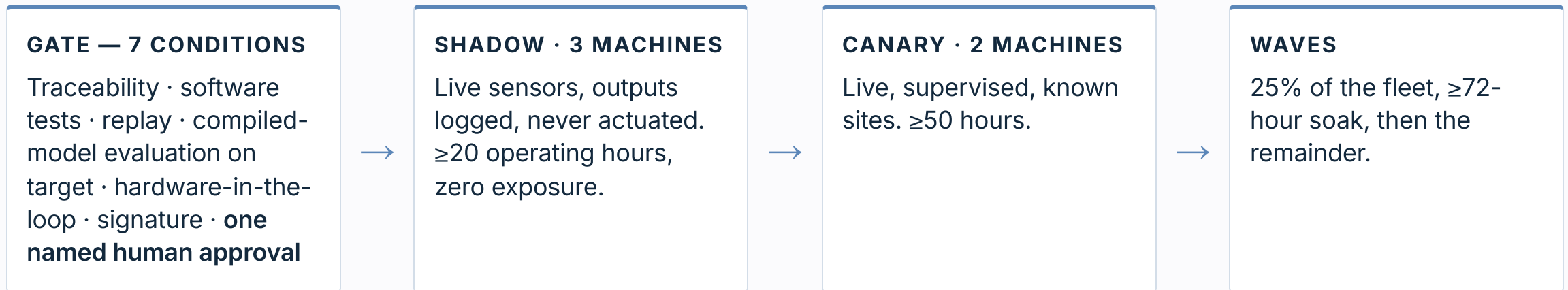
Why one unit. A model's accuracy depends on pre- and post-processing that live in the software. Change the normalisation constants or the NMS threshold and the model behaves differently **without the model changing**. A model validated against software v12 has no established behaviour under v13.

The compilation gap

The artifact that trains is not the artifact that runs. It is exported, quantised and compiled for the vehicle's accelerator, and quantisation degrades low-contrast inputs disproportionately — exactly the rare, costly slices. In this design **v7 passed dust at FP32 and failed it at INT8**. So the compiled engine is re-evaluated on target hardware.

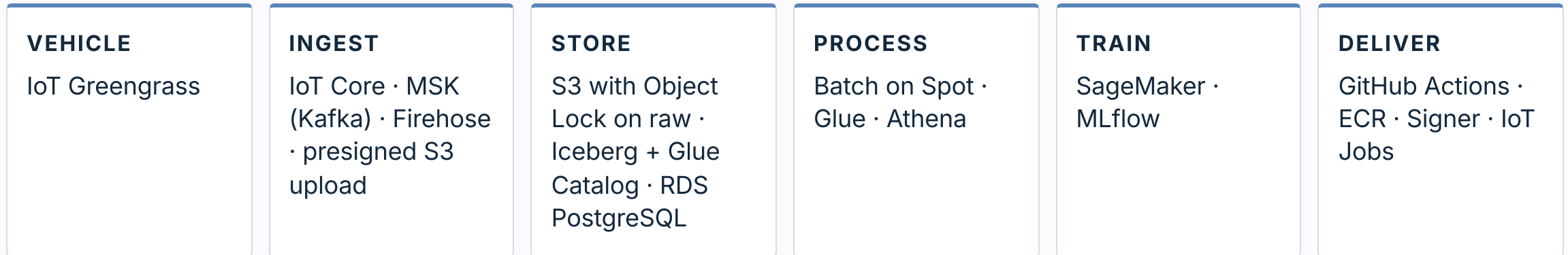
Cost One bundle couples release cadence — a one-line fix requires a full release and full validation. A separate schema-validated configuration path exists so threshold changes do not create pressure to bypass the process.

A seven-condition gate, then a staged rollout that reverts itself



- **The gate is enforced twice** — in the pipeline, and again on the vehicle. The two fail independently: a pipeline can be bypassed with sufficient access, but a vehicle that refuses unsigned bundles cannot be made to accept one remotely.
- **Dual-slot A/B with automatic revert on a failed boot health check** — rollback works with **no network and no human**, which is exactly the state a bad bundle creates.
- **Bundle version is recorded in every recording's session metadata.** Data cannot be rolled back; a defective model that ran six hours produced six hours of recordings

Cloud: managed by default, and two rules behind it



- The scarcest resource is not compute or storage. It is engineering attention.
- **Buy managed unless managed forces a design compromise.** Kafka is the exception bought deliberately — mining sites will have no dependable cloud link, and the streaming layer is the one component that must then run locally.
 - **Build only the selection and curation logic.** Everything else is plumbing, and plumbing should be rented.
 - **Two orchestrators, one rule.** Step Functions for thousands of short per-recording verifications; Airflow for the dependency-rich, backfillable pipeline DAGs.

What breaks first is not the cloud

BINDS FIRST	WHY	RESPONSE
Depot upload bandwidth	A link serving 5 machines does not serve 50	More aggressive on-vehicle reduction; the physical tier expands
The annotation budget	Ten times the data is not ten times the budget	Selection carries more weight — its compounding argument
Bench hardware for hardware-in-the-loop	Scales with vehicle variants , not fleet size	Per-variant benches; parallel compile-and-evaluate stages

Evolution path, in the order it should happen: per-variant bundles when verticals multiply → a learned ranker when hand-tuned weights stop being obviously right → expanded pre-labelling when annotation throughput binds → distributed training when datasets outgrow a single node → Kafka moved on-prem when customer sites appear.

Sized for tens of machines now and low hundreds within three years — explicitly not 10,000. Every place that assumption binds is named, along with the trigger that changes the decision.



The three edges that close the loop

① **Field failures → retraining.** A deployed model's slice-level failure rate crosses a threshold and triggers the pipeline that replaces it. The deployed model's own failures fire its replacement.

② **Weak evaluation slices → next collection.** A slice that scores badly raises its weight in the next selection cycle. Evaluation stops being a report and becomes an input.

③ **Shadow disagreements → selection pool.** A frame on which two independently trained models disagree is difficult by construction. [Validating a model produces the data that improves the next one.](#)

Why the returns compound rather than repeat

- As the model improves, the share of footage it already handles grows — so **the value of a randomly chosen hour falls every cycle**. Selection must get smarter simply to hold its value constant.
- The production model pre-labels clips before annotation, so **a better model makes annotation cheaper** — the same budget buys more labelled hours each cycle.
- Every release contributes new regression tests and new replay sessions, so **the test suite**

Five trade-offs, with the cost stated

CHOSE	OVER	COST ACCEPTED
Depot WiFi for ~98% of bytes	One upload path	Depends on a daily depot return. Physical media is the named fallback
Four selection signals	Uncertainty sampling alone	More scoring machinery to build and maintain
Session-level sticky splits	Random frame split	Less data-efficient; splits cannot be balanced precisely
The bundle as the release unit	Independent model and software deploys	Couples release cadence — a one-line fix needs full validation
Statistical reproducibility, stated	Bitwise determinism	Re-runs vary within normal run-to-run variance

The full register is 47 entries, each stated as **what was chosen · what it was chosen over · what it costs** — because identified trade-offs matter more than the stack selected.



Three claims this design makes

It is a flywheel, not a pipeline. Three feedback edges are named and mechanised. Remove them and this is a pipeline that is run repeatedly.

Every gate fails closed. No stage produces output by default. A label batch, a dataset version, a model candidate, a release bundle — each exists only because a specific check passed, and the failed attempt is recorded as completely as the successful one.

It is sized for this company, and it says where it stops. Every place that assumption binds is named, along with the trigger that would change the decision.

Thank you. Happy to go deeper on selection, the release gate, or the cost model.